

ThoughtBook

Follow / Unfollow Feature

Complete Development Workflow: Database → FastAPI → React → Private Profiles → Notifications → WebSocket → Feed → Testing → Pull Request

Project	ThoughtBook
Backend	FastAPI + PyMySQL
Database	Aiven MySQL
Frontend	React + Vite
Authentication	JWT in HttpOnly cookie
Existing privacy field	PRIVATE_PROFILE
Feature	Follow / Unfollow

Core rule: implement one layer at a time. Finish and test the backend relationship before building the React button.

1. Complete Roadmap

Phase	Work	Output
1	Functional design	One-way follow model; public/private behavior
2	Database	follows table, constraints, indexes
3	Database layer	app/database/follows.py
4	FastAPI	Follow, unfollow, status, lists, requests
5	Swagger testing	Backend verified before React
6	React profile	Button + counts
7	Followers/following	Lists and user cards
8	Private profiles	Pending / accept / reject
9	Notifications	Persistent follow/request notifications
10	WebSocket	Real-time notification delivery
11	Feed	Posts from followed users
12	Hardening	Security, edge cases, tests, PR

Development order: database → DB functions → FastAPI → Swagger → React → private requests → notifications → WebSocket → feed.

2. Functional Design

2.1 One-way relationship

A follow is directional. If A follows B, B does not automatically follow A.

User A ---- follows ----> User B
A's following contains B.
B's followers contains A.
B following A is a separate relationship.

2.2 Public profile

POST /follow/{target_user_id}
A follows public B
↓
status = accepted
↓
A is immediately a follower of B

2.3 Private profile

POST /follow/{target_user_id}
A follows private B
↓
status = pending
↓
B accepts → accepted
B rejects → relationship removed

2.4 Unfollow

Unfollow removes an accepted A → B relationship. It is not a block and does not automatically affect other relationships.

2.5 Button states

- Not following → Follow
- Following → Following (with Unfollow action)
- Pending → Requested

- Own profile → no Follow button

3. Database Design

3.1 follows table

```
CREATE TABLE follows (
    follow_id INT PRIMARY KEY AUTO_INCREMENT,
    follower_id INT NOT NULL,
    following_id INT NOT NULL,
    status ENUM('pending', 'accepted') DEFAULT 'accepted',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unique_follow (follower_id, following_id),
    FOREIGN KEY (follower_id) REFERENCES user_log_details(user_id),
    FOREIGN KEY (following_id) REFERENCES user_log_details(user_id)
);
```

3.2 Column responsibilities

- follow_id: unique relationship/request ID.
- follower_id: user initiating the follow.
- following_id: user being followed.
- status: pending for a private-profile request; accepted for active following.
- created_at: relationship creation time.
- UNIQUE KEY: prevents duplicate A → B relationships.
- Foreign keys: keep relationships tied to real users.

3.3 Rules

- Reject self-follow.
- Count only accepted relationships as followers/following.
- Do not create a second pending request for the same pair.
- Use parameterized PyMySQL queries.

4. Database Layer — app/database/follows.py

Keep SQL operations separate from FastAPI routes, matching the project's existing database-layer pattern.

Function	Purpose
create_follow	Create accepted follow or pending request
unfollow_user	Remove accepted relationship
get_follow_status	Check A → B state
get_followers	Return accepted followers
get_following	Return accepted following
get_follower_count	Count accepted followers
get_following_count	Count accepted following
get_pending_requests	Return requests for current user
accept_follow_request	pending → accepted
reject_follow_request	Delete pending request

4.1 Follow algorithm

1. Get current user from JWT.

2. Validate target user exists.
3. Reject current user == target user.
4. Check whether $A \rightarrow B$ already exists.
5. Read B's PRIVATE_PROFILE value.
6. Public $\rightarrow$ insert accepted.
7. Private $\rightarrow$ insert pending.
8. Commit.
9. Return status.

4.2 Unfollow algorithm

1. Authenticate current user.
2. Validate target.
3. Find $A \rightarrow B$.
4. Remove the relationship.
5. Commit.
6. Return success.

4.3 Counts

Follower and following counts should normally include only status = accepted.

5. FastAPI API Layer

5.1 Route file

```
backend/
  routes/
    follows.py
```

Method	Endpoint	Purpose
POST	/follow/{user_id}	Follow or send request
DELETE	/follow/{user_id}	Unfollow
GET	/follow/status/{user_id}	Check relationship
GET	/users/{user_id}/followers	Followers list
GET	/users/{user_id}/following	Following list
GET	/users/{user_id}/follow-counts	Counts
GET	/follow/requests	Pending requests
POST	/follow/requests/{follow_id}/accept	Accept
DELETE	/follow/requests/{follow_id}/reject	Reject

5.2 Example responses

```
Follow public:
{
  "message": "Follow successful",
  "target_user_id": 25,
  "status": "accepted"
}

Follow private:
{
  "message": "Follow request sent",
  "target_user_id": 25,
  "status": "pending"
}

Status:
{
  "is_following": true,
  "is_followed_by": false,
  "status": "accepted"
}
```

6. Authentication and Authorization

```

React
|
| fetch(..., { credentials: "include" })
v
FastAPI
|
| read access_token HttpOnly cookie
v
decode JWT
|
v
current_user_id

```

Critical: never accept follower_id from React. The client sends only the target user ID. The backend derives follower_id from the authenticated JWT.

- All mutation endpoints require authentication.
- Accept/reject is allowed only for the owner of the target profile.
- Frontend route protection is UX; backend authorization is the real security boundary.
- Use 401 for missing/invalid authentication and controlled 4xx responses for invalid actions.

7. Swagger / Backend Testing

Complete backend testing before writing the Follow button.

1. Create two test users A and B.
2. Authenticate as A.
3. GET /follow/status/B.
4. POST /follow/B.
5. Verify the follows row.
6. Verify status = accepted for a public B.
7. Verify follower/following counts.
8. DELETE /follow/B.
9. Verify the row is removed and counts change.
10. Make B private.
11. POST /follow/B again as A.
12. Verify status = pending.
13. Authenticate as B.
14. GET /follow/requests.
15. Accept A's request.
16. Verify pending → accepted.
17. Repeat with Reject and verify the row is removed.

8. React Integration

8.1 Component structure

```

ProfilePage
  ■■■ FollowButton
  ■ ■■■ GET /follow/status/{profileUserId}
  ■ ■■■ POST /follow/{profileUserId}
  ■ ■■■ DELETE /follow/{profileUserId}
  ■■■ FollowCounts
  ■■■ Followers view
  ■■■ Following view

```

8.2 Profile load

```
GET /follow/status/{profileUserId}
GET /users/{profileUserId}/follow-counts
```

The UI should render backend state rather than guessing whether the profile is followed.

8.3 Follow flow

```
Click Follow
↓
POST /follow/{id}
↓
accepted → Following
pending  → Requested
error    → keep previous state + show error
```

8.4 Unfollow flow

```
Click Unfollow
↓
DELETE /follow/{id}
↓
status = not following
↓
button = Follow
↓
refresh counts
```

9. Followers and Following Lists

9.1 Followers query

```
SELECT u.user_id, u.user_name, u.user_profile_pic
FROM follows f
JOIN user_log_details u ON u.user_id = f.follower_id
WHERE f.following_id = %s
      AND f.status = 'accepted'
ORDER BY f.created_at DESC;
```

9.2 Following query

```
SELECT u.user_id, u.user_name, u.user_profile_pic
FROM follows f
JOIN user_log_details u ON u.user_id = f.following_id
WHERE f.follower_id = %s
      AND f.status = 'accepted'
ORDER BY f.created_at DESC;
```

Your profile can display: Followers | Following. Clicking either value opens a list of users. Each list item can show profile picture, name, and the appropriate Follow/Following state.

10. Private Profiles

```
A follows private B
↓
pending
|
+---+---+
|       |
Accept  Reject
|       |
↓       ↓
accepted deleted
```

10.1 Request authorization

Only B can accept or reject A's request. The backend must verify that the current authenticated user is the following_id of the request.

10.2 Request endpoints

```
GET    /follow/requests
POST   /follow/requests/{follow_id}/accept
DELETE /follow/requests/{follow_id}/reject
```

11. Notifications

First implement persistent notifications with MySQL. Add WebSocket delivery only after the HTTP/database notification system works.

```
CREATE TABLE notifications (
  notification_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  sender_id INT,
  type VARCHAR(50) NOT NULL,
  message VARCHAR(255),
  reference_id INT,
  is_read BOOLEAN DEFAULT FALSE,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES user_log_details(user_id)
);
```

11.1 Events

Public follow:

A → B

■■■ follows row = accepted

■■■ notification for B = "A followed you"

Private request:

A → B

■■■ follows row = pending

■■■ notification for B = "A requested to follow you"

B accepts:

■■■ pending → accepted

12. WebSocket — Later

```
React
|
| WebSocket
v
FastAPI
|
|--> save notification to MySQL
|
|--> send real-time event to connected recipient
|
v
React notification UI
```

- Database remains the source of truth.
- Offline users still have notifications stored in MySQL.
- HTTP APIs are easier to debug before introducing WebSocket.
- WebSocket is the delivery mechanism, not the relationship database.

13. Feed Integration

```
Current user
↓
accepted following relationships
↓
followed user IDs
↓
posts by those users
↓
feed
```

Start with a simple chronological feed of followed users. Recommendation logic can be added later.

14. Edge Cases

Case	Expected behavior
Follow yourself	Reject
Target doesn't exist	404
Already following	No duplicate row
Already pending	No duplicate request
Unfollow someone not followed	Controlled response
Accept another user's request	Reject
Reject another user's request	Reject
No/invalid JWT	401
Private profile	pending
Public profile	accepted
Own profile	No Follow button

15. Testing Checklist

15.1 Database

- Valid accepted follow inserts.
- Duplicate follow is prevented.
- Self-follow is rejected.
- Unfollow removes relationship.
- Pending request is created for private profile.
- Accept changes pending to accepted.
- Reject removes pending relationship.
- Counts include only accepted rows.

15.2 API

- Authenticated follow succeeds.
- Unauthenticated mutation returns 401.
- Status endpoint is correct.
- Followers/following contain only accepted rows.
- Accept/reject checks ownership.
- Invalid IDs return controlled errors.

15.3 React

- Own profile hides Follow.
- Public profile shows Follow initially.
- Follow changes state without full reload.
- Reload preserves correct state.
- Unfollow returns to Follow.
- Private profile shows Requested.

- Counts update after mutation.
- Network errors do not corrupt state.

16. Git Workflow for This Feature

```
git checkout main
git pull origin main

git checkout -b follow-feature

# implement + test

git add .
git commit -m "Add follow database and API"

# continue
git add .
git commit -m "Add follow and unfollow UI"

git push -u origin follow-feature
```

16.1 Pull Request

```
follow-feature
  |
  | push
  v
GitHub
  |
  | Pull Request
  v
main
```

Use your existing protected-main workflow: develop on the feature branch, push it, open a Pull Request into main, review/test, then merge the PR.

16.2 Suggested commits

- Create follows table
- Add follow database functions
- Add follow FastAPI endpoints
- Add follow API validation
- Add profile follow button
- Add followers and following lists
- Add private profile requests
- Add follow notifications
- Add follow feature tests

17. Security Checklist

- JWT remains in an HttpOnly cookie.
- React uses credentials: 'include'.
- Backend derives follower_id from JWT.
- Target user ID is validated.
- Self-follow is rejected.
- Duplicate relationships are prevented.
- Accept/reject checks the current user's ownership.

- All SQL uses parameterized queries.
- Frontend protection is not treated as API security.
- Production uses HTTPS and secure cookies.

18. Deployment Checklist

- Create the follows table in the production Aiven database.
- Verify production database SSL/configuration.
- Register the FastAPI follow router.
- Verify production CORS and credentials.
- Use secure cookies over HTTPS.
- Set frontend API URL to production backend.
- Test with two production test accounts.
- Verify public and private profile behavior.
- Check indexes/query performance.
- Merge only through the Pull Request workflow.

19. Definition of Done

1. follows table exists with constraints.
2. Self-follow and duplicate follow are prevented.
3. JWT determines the current user.
4. Follow works.
5. Unfollow works.
6. Follow status works.
7. Follower and following counts work.
8. Followers and following lists work.
9. Public profiles create accepted relationships.
10. Private profiles create pending requests.
11. Accept/reject is authorized correctly.
12. React displays Follow / Following / Requested correctly.
13. UI updates after mutations.
14. Errors are handled.
15. Notifications persist if included in the milestone.
16. Important API/UI edge cases are tested.
17. Feature is merged via Pull Request.

20. Exact Build Plan for Your Current Project

Session	Task	Done when
1	Create follows table	MySQL table + constraints verified
2	Build app/database/follows.py	DB functions work
3	Build app/routes/follows.py	Routes appear in Swagger

Session	Task	Done when
4	Test backend	Public follow/unfollow works
5	Add status/counts	Profile gets correct relationship
6	Build Follow button	React mutation works
7	Build lists	Followers/following are correct
8	Private requests	Pending/accept/reject works
9	Notifications	Follow/request events persist
10	WebSocket	Real-time delivery works
11	Feed	Followed-user posts can be selected
12	Testing + PR	Feature is ready to merge

Immediate next step: create and verify the follows table. After that, implement the database functions. Do not start with React until the relationship APIs are working in Swagger.